Lesson 1.1: Foundations of full-stack architecture

What you learn

- **Client-server model.** A client (browser or app) sends a request; a server processes it and sends a response. The server holds data and rules; the client shows them.
- **HTTP protocol.** Requests have a method (GET, POST, PUT, PATCH, DELETE), a path, headers and sometimes a body. Responses carry a status code (200 OK, 201 Created, 400 bad request, 401 not signed in, 404 not found, 500 server error), headers and a body.
- **RESTful APIs.** Resources have URLs (for example /users/1) and the HTTP method says the action. Responses are usually JSON. Each request carries what the server needs; the server keeps no hidden session between calls.
- **GraphQL.** One endpoint where the client asks for exactly the fields it needs. It reduces over-fetching but adds a schema, resolvers and caching to learn. REST is simpler for small APIs; GraphQL helps when many screens need different shapes of the same data.

Tools and resources (official sites, not checked live)

- MDN Web Docs: HTTP
- Postman documentation

ASSIGNMENT

Practice for lesson 1.1

Business problem: Prove you can read and make an HTTP exchange.

Steps

1. Run the tested server and client below.
2. Open the same kind of request in Postman against any public API.
3. Note the method, status code and content type of each response.

Expected output: A 200 response with a JSON body and a note of the three values.

SOLUTION (ANSWER KEY)

PYTHON

```
import threading, json, urllib.request
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        body=json.dumps({"id":1,"name":"Asha"}).encode()
        self.send_response(200); self.send_header("Content-Type","application/json");
self.send_header("Content-Length",str(len(body))); self.end_headers(); self.wfile.write(body)
    def log_message(self,*a): pass
s=HTTPServer(("127.0.0.1",0),H); port=s.server_port
threading.Thread(target=s.serve_forever,daemon=True).start()
r=urllib.request.urlopen(f"http://127.0.0.1:{port}/users/1")
print(r.status, r.headers["Content-Type"], json.loads(r.read()))
assert r.status==200
s.shutdown()
```

The output shows status 200, content type application/json and the parsed body. In Postman you will see the same three items in the response panel.

Check yourself

1. Which status code means 'created'?
   Answer: 201.
2. When might GraphQL beat REST?
   Answer: When many clients need different subsets of the same data and over-fetching is a real cost.

Lesson 1.1 of the **Development and IT** lessons. See this lesson and its official links on skillnavora.in · All chapters and lessons · Open the full lesson guide. Tools, prices and plans change; confirm on each official site.